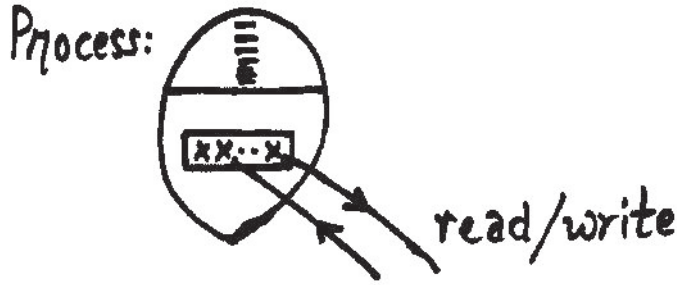
READ, WRITE CALLS data transfers



A File: [ø] [1] [2] .... [N-1] a logical picture

(a stream of bytes)

Assumption: Each call always does transfer of consecutive bytes.

Analogy: reading a book

We want: to evolve parametens of read & write.

Process data space: Data box can be

(a) built_in data type : consecutive bytes
(b) array : elements consecutive
(c) structure : members consecutive

Consecutive Bytes Transfer

Needs: First byte in the file
First byte in the process data space
Number of bytes to be transferred.

read, write calls No. of bytes

COBOL: works on records
always transfers fixed number of bytes for a given file.

C : we would like to transfer different number of bytes on different reads/writes.

Analogy: Eat a jamoon.

No. of bytes is a parameter here.

Process Address:

COBOL has record area specified at open time.

C : data can be transferred to possibly different areas at different calls.

Address of 1st byte in process space becomes another parameter.

File Position:

Eveny parameter has a cost: to be pushed & accessed

Observation: most accesses from beg. to end. occasionally otherwise.

Analogy: Buy metador van ?

read, write params File position

Method: Keep file position for next transfer in the open instantiation.

i.e. instead of: read n bytes
say : read next n bytes

open: sets file position to zero.

Hence, file position is not a parameter to read/write.
(like reading a book : read next page)

Exceptions: like

(a) go to end of book to find murderer
(b) skip couple of pages of sunset description
(c) re-read an interesting scene

to be handled as special cases.
(Hope for best but be prepared for the worst)

Special cases may take longer time. => okay.

NOTE: COBOL-like C : Add # of bytes & address of process space as open parameters.

read, write params

NOTE: Number of calls:

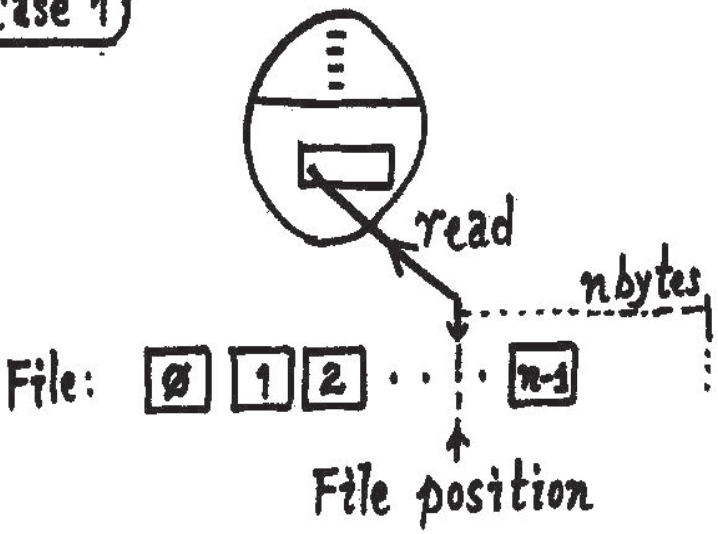
(a) 1 open
(b) N reads/writes (1/1m/1b)
(c) 1 close

Templates

```
int  read
     write (int fd, char *data, int nbytes);
```

read, write returns

Case 1



sufficient # of bytes are not available in the file for reading.

analogy: - buy a packet of cigarrettes
- can only buy 5 cigarrettes
Question: Success or Failure ?

read, write returns

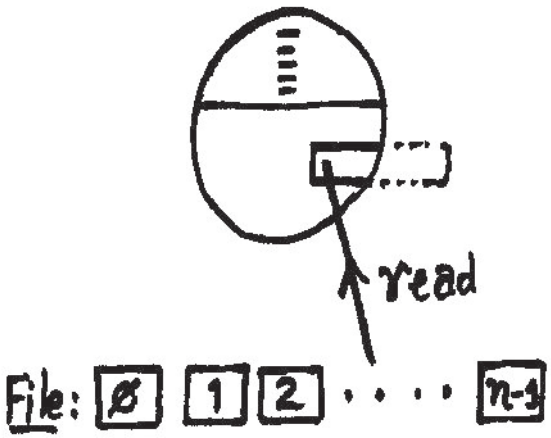
read/write returns # of bytes actually transferred.

Question: Buy one cigarette ⇒ Could buy no cigarettes
Success or failure ? success !

i.e. File position beyond the last byte of file.

case 2 write system call.
Fails when partition is full.
(no more data blocks)
(create fails when no more inode blocks)
Use df (1) command to look at free blocks.

case 3



read/write executing with process permissions.
Hence segmentation violation generated by m.m.u.
& process killed.

SAMPLE CALLS for read, write

sample 1 : To read one byte.

```
char x;
---
if (read (fd, & x, 1) != 1) {
   ------fprintf(stderr, "eof "); exit(0);
}
```

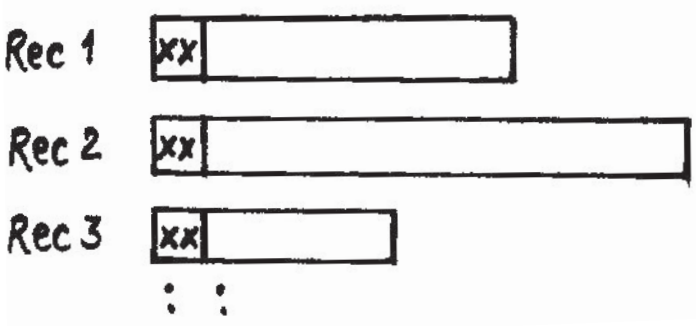
sample 2: To read an integer.

```
int y;
---
if (read (fd, (char *) & y, sizeof int)
        != sizeof int)  { ----
```

Sample 3: To read a structure ⇒ very similar.

A failure may indicate corruption of the file.

Variable length Records



Use multiple transfers:

(a) transfer 1st field

(b) transfer rest of the record.

CLOSE some people don't even use it

close (fd) ⇒ frees the row of file table.
needed when too many files need to be opened.

exit : automatically closes open instantiations.

discipline : close all opened files.

LSEEK for exceptional transfers

lseek : simply modifies file position in the row of file table (i.e. open instantiation)

template

long lseek (int fd, long distance, int whence)

whence : 0 ⇒ distance from beg of file (i.e. absolute position) : +ve

1 ⇒ distance relative to current position. distance : +ve/-ve

2 ⇒ distance relative to one byte beyond end : zero/-ve

lseek returns the eventual file position.

lseek examples

(1) lseek (fd, 0, 0) rewinds the file

File: 

↑ final file position

(2) lseek (fd, 0, 2)

File: 

↑ final position

returned value = size of file.

Other file inode info: permissions, timestamps,..
(called file status)

System Calls stat (2) fstat (2)

stat (2) operates on fd

fstat (2) operates on path

A structure : struct stat in stat.h
This is transferred to process data space.

FCNTL is for file control

Ex: as an afterthought on open flags.

fcntl to change open flags

Changes: (a) resetting a bit to ø.
(b) setting a bit to 1.

NOTE: ioctl (2) used for special action on devices

Template: int fcntl / ioctl (int fd, int cmd, int arg);

cmd example: FD_FORMAT
(defined in sys/fd.h)

arg: if many infos., use a pointer to a struct.

fcntl commands: F_GETF & F_SETF

F_GETF : data transfer from file table row to process data space

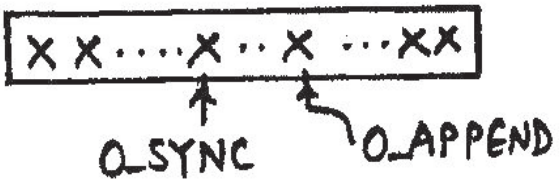
F_SETF : data transfer from process data space to file table row

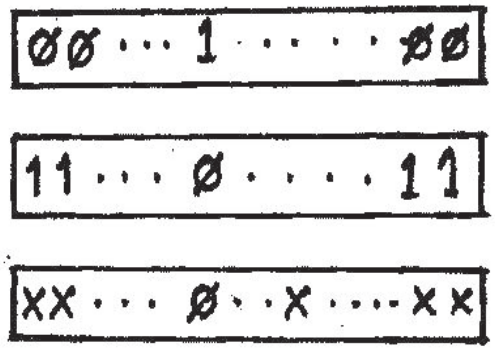
fcntl sample

To remove O_SYNC and set O_APPEND

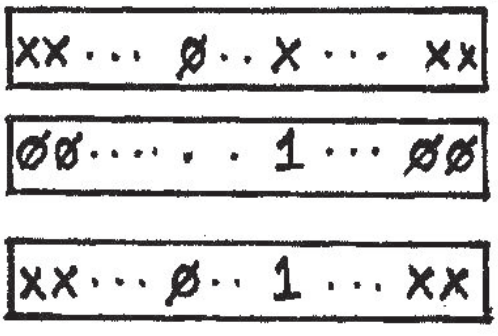
fcntl sample

```
int flags;
flags = fcntl (fd, F_GETF, 0);
```






```
i.e. flags &= ~O_SYNC;
```




```
i.e. flags |= O_APPEND;
fcntl (fd, F_SETF, flags);
```